

LangChain Interview Questions — Researched Reference (2026)

A note on how this was built

Every question in this document is grounded in something actually documented, discussed in practitioner communities, or reported in real technical writing — not invented from scratch. Sources are named inline. A few things worth knowing about how to read this:

- **Foundational questions** (what is LangChain, chain vs. agent, ReAct, tools) reflect genuinely standard interview content that shows up consistently across multiple independent sources — treat these as safe to prepare confidently.
 - **The security item (CVE-2025-68664)** is independently verified across five separate security research firms and the official GitHub Advisory Database — this is real and worth knowing cold if you're interviewing anywhere that runs LangChain in production.
 - **A few "practitioner anecdote" style claims** you'll see floating around online (specific anonymized Reddit threads citing exact percentage improvements, an exact "~15 tool call" degradation threshold) are single-source and unverifiable from where I sit — I've left those out rather than repeat them as settled fact. If you encounter them elsewhere, treat them as one person's experience, not an industry consensus.
-

Section 1 — Foundations (Junior Level)

Q1. What is LangChain, and what problem does it actually solve?

LangChain is an open-source framework for building applications powered by language models — it provides a standard interface for models, prompts, tools, and memory, plus a large ecosystem of integrations, so you're not hand-rolling provider-switching logic, tool-dispatch loops, and retrieval pipelines from scratch for every project.

Q2. What's the difference between a chain and an agent?

A **chain** is a deterministic sequence of steps, wired together before runtime, that runs the same way every time. An **agent** is non-deterministic — a model decides at runtime which tool to call next, based on what's happened so far, looping until it decides the task is done.

Q3. What is the ReAct pattern?

ReAct ("Reasoning + Acting") is the foundational pattern behind agent loops: the model alternates between a reasoning step ("Thought") and an action step ("Action" — usually a tool call), reading back the result ("Observation") before deciding what to do next. This loop repeats until the model produces a final answer instead of another action.

Q4. What is `AgentExecutor`, and why might an interviewer ask whether you still use it?

`AgentExecutor` is LangChain's original runtime wrapper for running the agent loop — it manages tool dispatch, tracks intermediate steps, and enforces iteration limits. As of LangChain v1.0 (October 2025), it's considered legacy: the current recommended path is

`create_agent`), built on LangGraph, which exposes state at every step for inspection and human approval in ways `AgentExecutor` was never designed for. An interviewer asking this is often checking whether you know the framework has moved on, not just whether you can recite how the old pattern works.

Q5. What does `.as_retriever()` do, and why does the retriever abstraction matter?

Any vector store becomes a `Runnable` retriever via `.as_retriever()`, accepting a `search_type` ("similarity", "mmr", or "similarity_score_threshold") and `search_kwargs`. Because every vector store implements the same retriever interface, swapping the underlying vector database is a one-line change rather than a rewrite.

Section 2 — Structured Output (Junior–Mid Level)

Q6. Why is `with_structured_output()` more reliable than asking a model to "please reply in JSON"?

Plain prompting relies on the model choosing to follow an instruction — it can still wander, add commentary, or produce syntactically invalid JSON. `with_structured_output()` gives the model an actual schema to conform to, validated by Pydantic (or a `TypedDict`/JSON Schema) before the result ever reaches your code — if the model doesn't structurally comply, it doesn't pass through as valid data.

Q7. What's the difference between a Pydantic `BaseModel` and a `TypedDict` for structured output?

A Pydantic `BaseModel` validates types at runtime — if a field is the wrong type, it raises an error. A plain `TypedDict` (or dataclass) defines the same shape but performs no runtime validation at all — lighter weight, but nothing stops a malformed value from getting through.

Q8. What are `ToolStrategy` and `ProviderStrategy`, and when would you choose one over the other?

Both are mechanisms for enforcing structured output at the agent level (`create_agent(response_format=...)`). `ProviderStrategy` uses a model provider's own native structured-output feature directly — fast, but only available where the provider supports it. `ToolStrategy` forces structure via a synthetic tool call, which works on any model with general tool-calling support, at a small overhead cost. When you don't specify either, LangChain auto-selects based on the model's declared capability profile.

Q9. What happens when a model's structured output violates a schema constraint (like a numeric range)?

The validation error is fed back to the model as a message, and the model retries with the exact constraint violation named — this happens automatically through the normal agent loop, with no custom retry code required. This behavior is configurable via `handle_errors` on `ToolStrategy`.

Q10. How would you design a schema for a system where an incoming message could be one of several distinct intents (e.g., book, cancel, modify)?

Use a `Union` of separate Pydantic schemas as the `response_format` — the model itself decides which schema fits the message, and your code checks which one it received (typically with `isinstance()`) before acting on type-specific fields.

Section 3 — Tools and Agents (Mid Level)

Q11. What makes a good tool docstring, and why does it matter more than it looks like it should?

The docstring is the primary text the model reads to decide *whether* and *when* to call a tool — not documentation for the next engineer reading the code. A vague docstring produces a model that either never uses the tool or uses it at the wrong moments; a precise one, describing exactly when the tool applies, produces reliable tool selection.

Q12. What's the difference between binding a tool and actually executing it?

`.bind_tools()` only makes a model aware that a tool exists and lets it *request* a call — it does not run anything. Actually executing the requested tool and feeding the result back for a final answer is `create_agent`'s job, not the bare model's.

Q13. What does `ToolRuntime` give a tool access to, and why is it hidden from the model?

`ToolRuntime` (accepted as a special `runtime` parameter) gives a tool access to `state` (short-term, this conversation), `context` (immutable per-run data), and `store` (long-term memory across sessions) — all invisible to the model, because LangChain recognizes the `ToolRuntime` type annotation and excludes it from the schema the model sees. This means the model never tries to guess a value for it.

Q14. What's the difference between an agent's `thread_id` and its `context`?

`thread_id` scopes conversation history — it persists automatically across calls that share the same ID, via a checkpoint. `context` carries per-run data (like a user ID or session details) that must be passed fresh on every single invocation — it does not automatically carry over the way message history does. Conflating the two is a common real-world bug once an application has multiple users.

Q15. How would you restrict a tool so only certain users can access it — and why is a system-prompt instruction not good enough?

Telling the model "don't offer this to non-premium users" in the system prompt relies on the model choosing to follow that instruction — a clever or unlucky phrasing can still get around it. The stronger approach uses `wrap_model_call` middleware to filter the tool list itself before the model ever sees it, based on runtime state — the tool is never an option at all, not merely discouraged.

Q16. What's a multi-tool agent, and what's a practical risk of giving an agent too many tools at once?

A multi-tool agent has access to several tools and must choose the right one(s) per query rather than being limited to a single function. The practical risk (echoed across several 2025–2026 practitioner discussions, though exact thresholds vary by task and model) is that too many available tools can degrade the model's decision quality — it's a real reason multi-agent patterns (specialized agents, each with a narrower toolset) exist, rather than one agent holding every capability.

Section 4 — Production and Senior-Level Questions

Q17. When would you specifically NOT reach for LangChain?

A genuinely defensible senior answer: skip it (or a heavy abstraction layer generally) when a task is simple and single-step with no chaining or memory need, when raw latency matters more than developer convenience, or when a narrower, purpose-built model would outperform an LLM outright — classification tasks like sentiment analysis or named-entity recognition are a commonly cited example, where a small encoder-based model beats an LLM chain on speed, cost, and accuracy simultaneously.

Q18. What is LangSmith, and what does "tracing" actually give you that print statements don't?

LangSmith records a full, timestamped trace of every model call, tool call, and — with middleware — every intermediate hook, for a specific agent run. Unlike ad hoc print statements (which only show what you thought to check), a trace shows the complete, real sequence of what happened, which is often the only way to actually diagnose a non-deterministic agent failure.

Q19. What is RAGAS, and what does it measure that a simple accuracy check doesn't?

RAGAS (Retrieval-Augmented Generation Assessment) is an evaluation framework scoring RAG systems across several distinct dimensions — commonly cited ones include whether an answer actually addresses the question, whether every claim is grounded in retrieved context (a direct proxy for hallucination), and whether retrieval itself surfaced the right passages. The point of naming multiple metrics instead of one "accuracy" number is that a RAG system can fail in different, independent ways — being well-written but ungrounded, or grounded but missing the actual answer.

Q20. There's a real, disclosed vulnerability in `langchain-core` — do you know it?

Yes — **CVE-2025-68664** (disclosed December 2025, CVSS 9.3), a serialization injection vulnerability in `langchain-core`'s `dumps()`/`dumpd()` functions. The functions failed to escape a reserved internal marker key (`"lc"`) when serializing free-form dictionaries, meaning attacker-controlled data (reachable via prompt injection into fields like model response metadata) could be misinterpreted as a trusted internal object during deserialization — potentially exposing environment-variable secrets. It's patched in `langchain-core >= 0.3.81` and `>= 1.2.5`. An interviewer asking this is checking

whether you track security advisories for the frameworks you ship, not just whether you can use them. (*Independently verified via Snyk, Orca Security, The Hacker News, and the official GitHub/GitLab Advisory Databases.*)

Q21. How would you approach debugging a production agent that appears to be hanging?

A systematic answer beats "I'd restart it": check whether tracing (LangSmith or equivalent) is already on and look at the last completed step; if it's genuinely stuck, the most common causes are a tool call blocked on a slow or unresponsive external call with no timeout, or an agent that never produces a stopping condition because no iteration cap was set. The production-hardened fix is setting an explicit tool-call ceiling (via middleware, covered earlier in this course) and wrapping external calls in real timeouts — not discovering the lack of both live, under pressure.

Section 5 — Quick-Fire Conceptual Round

Good for a rapid warm-up before a real interview — answer each in one or two sentences.

1. What's the difference between `.invoke()`, `.stream()`, and `.batch()`?
 2. Why does `init_chat_model("provider:model")` matter more than it looks like it should?
 3. What's the difference between `model.with_structured_output()` and `create_agent(response_format=...)`?
 4. Name two of the four built-in `search_type` options a retriever supports, and what each does differently.
 5. What's the actual difference between short-term and long-term memory in an agent system?
 6. Why would you ever choose `TypedDict` over a Pydantic `BaseModel` for structured output?
 7. What does a checkpointer actually do, and what breaks if you forget to attach one?
 8. Name one thing `ToolMessage.artifact` can carry that `.content` shouldn't.
-

Sources referenced in this document

- `docs.langchain.com` — overview, philosophy, agents, tools, structured-output, retrieval pages (2026)
- GeeksforGeeks — "Agents and Tools in LangChain" (2025)
- Multiple independent security research write-ups on CVE-2025-68664: Snyk, Orca Security, The Hacker News, SOCRadar, Upwind, and the official GitHub/GitLab Advisory Databases (Dec 2025-Jan 2026)
- Community technical writing on ReAct/AgentExecutor migration patterns (Medium, DEV Community, 2025-2026)
- Exponent and Index.dev LangChain interview question banks (2026) — used for general question framing/topic coverage, not for any single quoted claim

